SYMPRAXIS CONSULTING

Azure Functions: Real-World Scenarios and Node.js Implementation Overview

Julie Turner

SYMPRAXIS CONSULTING



julieturner.net/me

Julie Turner

Partner / CTO · Sympraxis Consulting

- Working with SharePoint since **2007**
- **Microsoft MVP**, Office Apps & Services since 2017
- M365 & Power Platform **Community Team** since 2019
- Open-source co-maintainer: **PnPjs** & **hTWOo**
- Co-host: ***Code. Deploy. Go Live.***

Agenda

What are they?

What are they good for?

Common Architecture

Security

Configuration & Deployment

Q&A

What are they?

- Azure serverless compute
- Run small pieces of code, or "functions," in the cloud without being responsible for the infrastructure.
  - Serverless
  - Event-Driven
  - Scalable
  - Multi-language support

What are they good for?

- Triggered when items in Microsoft 365 change
  - Document drop library
  - Provisioning sites
  - Approval workflows
- Timer Jobs
  - Updating location fields using maps api
  - Synchronizing data (could also be real time)
  - Sending notification emails
- SPFx APIs with OBO flow
  - Elevating permissions through controls code flow

What are they good for?

- **HTTP Trigger**: Executes in response to HTTP requests. This is useful for creating APIs and webhooks.
- **Timer Trigger**: Runs on a specified schedule, similar to a cron job. Ideal for periodic tasks like data cleanup.
- **Queue Trigger**: Activated by new messages in an Azure Storage Queue. Useful for processing background tasks.
- **Event Grid Trigger**: Responds to events from Azure Event Grid, such as resource changes or custom events.
- **Blob Trigger**: Fires when a new or updated blob is detected in Azure Storage. Great for processing files.
- **Service Bus Trigger**: Triggered by messages in an Azure Service Bus queue or topic. Suitable for enterprise messaging scenarios.
- **SQL DB Trigger**: Executes when there are changes in an SQL DB collection. Useful for real-time data processing (doesn't seem to work on consumption plans ATM)
- **OTHERS**: Cosmos DB, Service Bus, Event Hub, IoT, SignalR, Durable Functions orchestrations, RabbitMQ triggers, Kafka triggers

What types are there?

- Consumption/Flex Consumption:
  - Scales dynamically based on demand.
  - You pay only for the time your functions are running.
- Premium:
  - EP1, EP2, EP3: More powerful instances with pre-warmed workers to reduce cold starts.
  - Support virtual network connectivity.
  - Best for Event-driven workloads
  - Pay for the number of core seconds and memory used
- Dedicated (App Service) Plan:
  - Runs on regular App Service plan rates.
  - Suitable for long-running functions and scenarios where you need more control over the infrastructure.
  - Best for Consistent workloads or long-running processes.
  - Pay for the VM instances regardless of usage

Common Architectures

HTTP + Queue

Event Grid + Queue



SYMPRAXIS CONSULTING

“Event” plus Queue

- Keeps the code that receives the event as simple as possible to avoid error conditions losing data integrity
- Failed queue items can be configured to reprocess automatically to ***n*** retries on ***t*** interval.
- Poison queue allows reprocessing of failed queue items.
- Allows you to manually processes events.

Security

- EntraId App Registrations/ MSAL
- Managed Identity/ Azure Identity
- OBO Flow

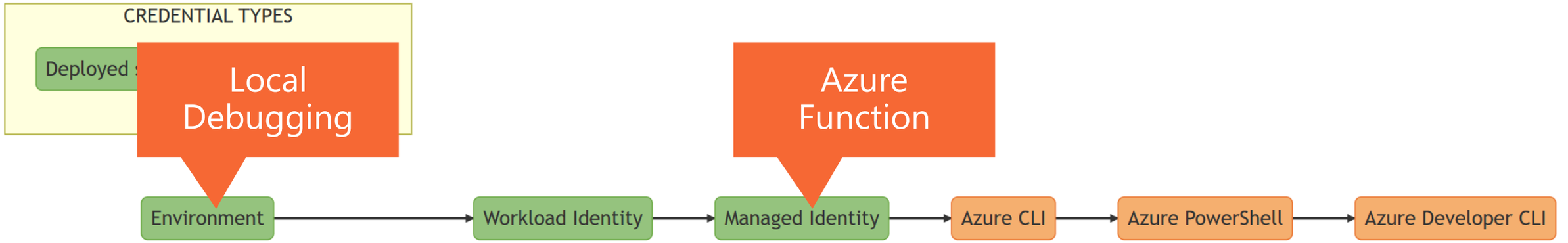


SYMPRAXIS
CONSULTING

@Azure/Identity

DefaultAzureCredential

- Environment - account information specified via environment variables and use it to authenticate.
- Workload Identity - deployed to Azure Kubernetes Service with Managed Identity enabled
- Managed Identity - deployed to an Azure host with Managed Identity enabled
- Azure CLI - developer has authenticated an account via the Azure CLI az login command
- Azure PowerShell - developer has authenticated using the Azure PowerShell module Connect-AzAccount command
- Azure Developer CLI - developer has authenticated an account via the Azure Developer CLI azd auth login comman

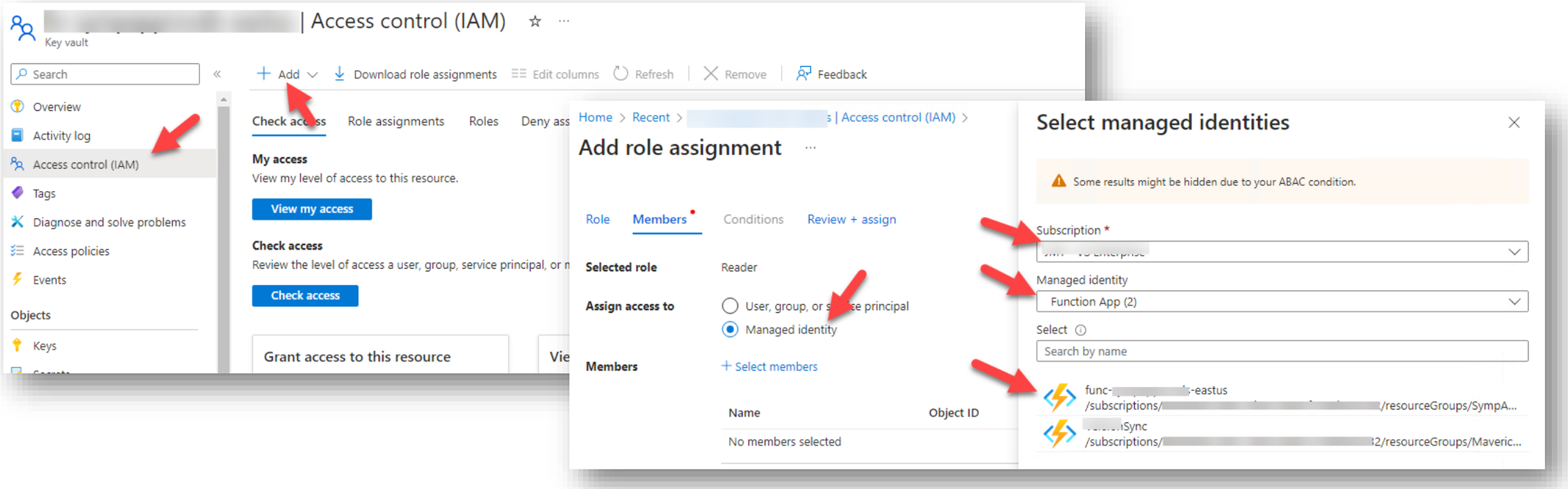


Managed Identity

Managed identities in Azure are a service that allows Azure resources to authenticate cloud services without the need for storing credentials in code or configuration files.

- You don't need to manage credentials.
  - Credentials aren’t even accessible to you.
- You can use managed identities to authenticate to any resource that supports Microsoft Entra authentication, including your own applications.
- Managed identities can be used at no extra cost.
- System-assigned
  - 1:1 relationship with the azure resource and its lifecycle is tied to the resource
- User-assigned
  - 1:many relationship to azure resources
- You authorize a managed identity to have access to one or more services

Authorizing a managed identity




```
m365 login --authType browser

m365 aad approleassignment add --appObjectId "1022615c-4433-4731-a933-53a9d2770e76" --resource "Microsoft Graph" --scopes "Files.ReadWrite.All,Group.
Read.All,Mail.Send,User.Read.All"

m365 aad approleassignment add --appObjectId "1022615c-4433-4731-a933-53a9d2770e76" --resource "SharePoint" --scopes "Sites.FullControl.All"

m365 aad approleassignment add --appObjectId "1022615c-4433-4731-a933-53a9d2770e76" --resource "SharePoint" --scopes "TermStore.ReadWrite.All"
```

OBO (On Behalf Of) Flow

- Request header will have authorization token
- Validate the token
- Optionally, get the user that made the request from the token



```
"dependencies": {
  "@azure/functions": "4.7.0",
  "@pnp/azidjsclient": "4.12.0",
  "@pnp/graph": "4.12.0",
  "@pnp/nodejs": "4.12.0",
  "@pnp/sp": "4.12.0",
  "@pnp/sp-admin": "4.12.0",
  "applicationinsights": "2.x",
  "jsonwebtoken": "^9.0.2",
  "jwks-rsa": "^3.2.0"
},
```

Validate

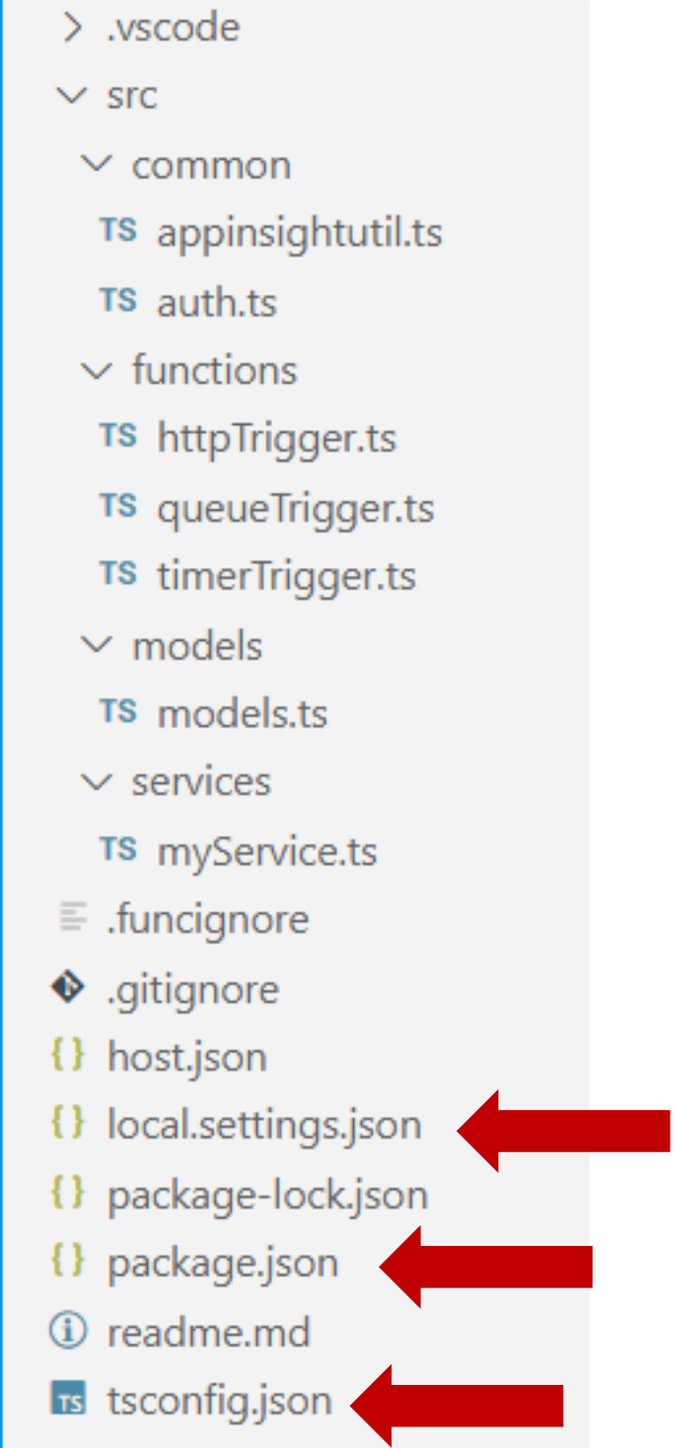
```
export async function requests(request: HttpRequest, context: InvocationContext): Promise<HttpResponseInit> {
  if (isNaN(Number(request.params.id))) {
    return { status: 400, body: JSON.stringify({ message: "Invalid request ID" }) };
  }
  const requestId = request.params.id;


  try {
    const tokenValidateService = new TokenValidateService(_apu);
    let validToken = false;
    if (process.env.DEBUG == 'true') {
      validToken = true;
    } else {
      validToken = await tokenValidateService.Validate(request.headers.get("authorization"));
    }
    if (validToken) {
```

Configuration & Deployment Demo

- New Project
- NodeJS – CommonJS vs ESModules setup
- Authentication
- Application Insights – logging
- The three most common triggers
  - HTTP
  - Queue
  - Timer
- Deploying & a word on BICEP

Project Folder



src	Source Code folder for solution
local.settings.json	Local debug settings for Azure Function
package.json	Package dependencies for the project
tsconfig.json	TypeScript configuration for transpiling

package.json

{} package.json > ...

```
{
  "name": "azurefuncintro",
  "version": "1.0.0",
  "description": "",
  "type": "module",
  ▷Debug
  "scripts": {
    "build": "tsc",
    "watch": "tsc -w",
    "clean": "rimraf dist",
    "prestart": "npm run clean && npm run build",
    "start": "func start",
    "test": "echo \"No tests yet...\""
  },
  "dependencies": {
    "@azure/functions": "4.16.0",
    "@azure/storage-queue": "^12.29.0",
    "@pnp/azidjsclient": "4.20.0",
    "@pnp/nodejs": "4.20.0",
    "@pnp/sp": "4.20.0",
    "@pnp/graph": "4.20.0",
    "applicationinsights": "2.x"
  },
  "devDependencies": {
    "@types/node": "20.x",
    "rimraf": "6.x",
    "typescript": "5.x"
  },
  "main": "dist/src/functions/*.js"
}
```

tsconfig.json

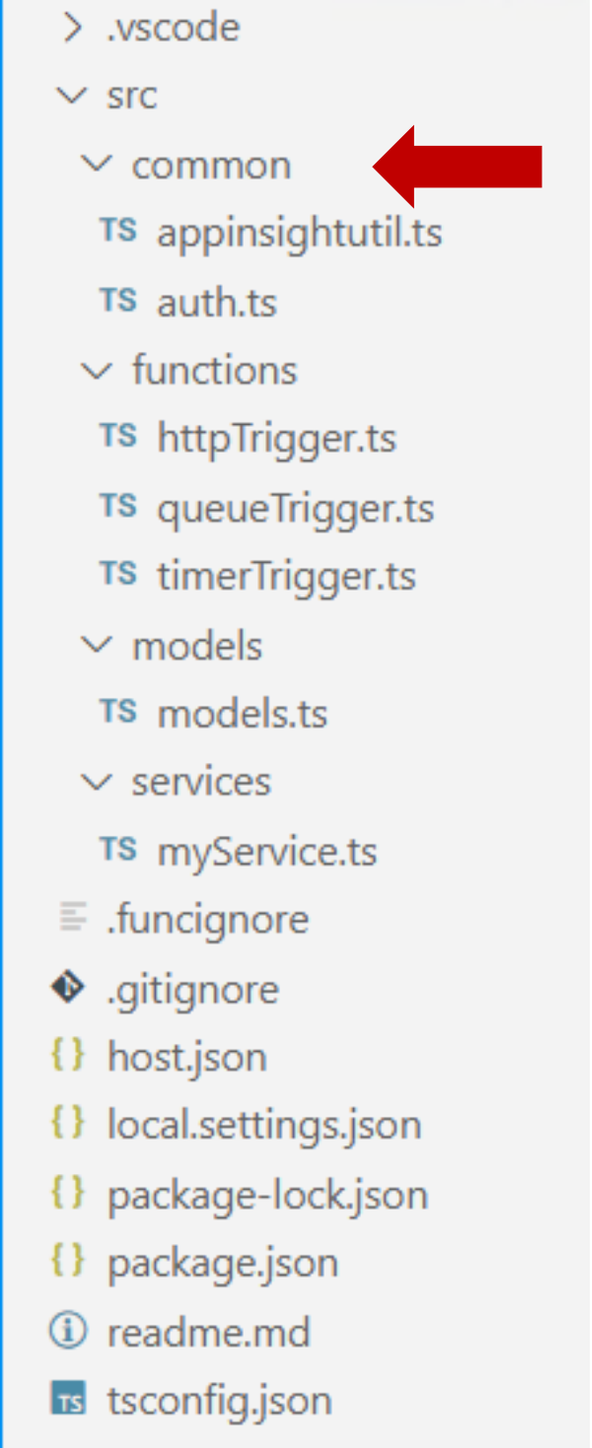
tsconfig.json > ...

```
1   {
2     "compilerOptions": {
3       "module": "ESNext",
4       "target": "ESNext",
5       "outDir": "dist",
6       "rootDir": ".",
7       "sourceMap": true,
8       "strict": false,
9       "moduleResolution": "bundler",
10      "allowSyntheticDefaultImports": true,
11      "typeRoots": [
12        "./node_modules/@types"
13      ],
14    }
15  }
```

local.settings.json

```
{
  "IsEncrypted": false,
  "Values": {
    "DEBUG": true,
    "APP_NAME": "SetProjectCoordinates",
    "APP_VERSION": "1.0.0",
    "APPINSIGHTS_LOGLEVEL": "0",
    "APPLICATIONINSIGHTS_CONNECTION_STRING": "InstrumentationKey=99999999-9999-9999-9999-999999999999;
    IngestionEndpoint=https://eastus-8.in.applicationinsights.azure.com/;LiveEndpoint=https://eastus.
    livediagnostics.monitor.azure.com/",
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "node",
    "AzureWebJobsFeatureFlags": "EnableWorkerIndexing",
    "SCHEDULE": "0 */5 * * * *",
    "MyQueuePath": "azurefunctionsintro",
    "Tenant": "contoso",
    "ProjectSite": "https://contoso.sharepoint.com/sites/MySite",
    "ProjectListId": "99999999-9999-9999-9999-999999999999",
    "AZURE_TENANT_ID": "99999999-9999-9999-9999-999999999999",
    "AZURE_CLIENT_ID": "99999999-9999-9999-9999-999999999999",
    "AZURE_CLIENT_CERTIFICATE_PATH": "C:\\private-key.pem"
  }
}
```

Project Folder



common	Helper class folder for solution
appinsightsutil.ts	Helper class to configure and log to Application Insights
auth.ts	Authentication helper class to create authenticated endpoints for PnPjs and other services

appinsightsutil.ts

```ts
export class AppInsightUtil implements IAppInsightUtil {
  private _uGuid = getGUID();
  private _commonProps = {};

  public Init(invocationId: string): void {
    this._initDefaultConfig(invocationId);
    AppInsights.start();
  }

  public AddCommonProperty(keyName: string, value: string): void {
    this._commonProps[keyName] = value;
    AppInsights.defaultClient.commonProperties = this._commonProps;
  }

  public Log(type: MessageType, msg: IAppInsightMessage): void {…
  }

  /**
 * Setup AppInsights with common settings
 *
 * @export
 */
  private _initDefaultConfig(invocationId: string): void {
    /* check if AppInsights already configured...
     *   > determined if `defaultClient` set & if `app.name` set
     *   > if not, bail on the config (don't double setup)
     */
    if (AppInsights.defaultClient?.commonProperties['app.name']) {
      return;
    }

    // enable stream of of live metrics; set auto dependency correlation
    AppInsights.setup(process.env.APPLICATIONINSIGHTS_CONNECTION_STRING).setSendLiveMetrics(true).setAutoDependencyCorrelation(true);

    // set the current tagged app & version of the function app
    this._commonProps["app.name"] = process.env.APP_NAME;
    AppInsights.defaultClient.commonProperties = this._commonProps;
    AppInsights.defaultClient.context.tags[AppInsights.defaultClient.context.keys.applicationVersion] = process.env.APP_VERSION;

    // set the operation id to the current invocation id
    AppInsights.defaultClient.context.tags[AppInsights.defaultClient.context.keys.operationId] = `${invocationId}-${this._uGuid.toString()}`;

  }
}
```

appinsightsutil.ts

```
4   export const SeverityLevel = AppInsights.Contracts.SeverityLevel;
5   export enum MessageType {
6     "Event",
7     "Exception",
8     "Metric",
9     "Trace"
10  }
11
12  export interface IAppInsightMessage {
13    logSource: string;
14    message?: string;
15    severity?: AppInsights.Contracts.SeverityLevel;
16    properties?: any,
17    name?: string,
18    exception?: Error
19  }
```





```
41    public Log(type: MessageType, msg: IAppInsightMessage): void {
42      if (AppInsights.defaultClient != undefined) {
43        if (msg.properties == null) { msg.properties = {}; }
          switch (type) {
45          case MessageType.Trace:
46            AppInsights.defaultClient.trackTrace({ message: msg.message, severity: msg.severity, properties: { source: msg.logSource, ...msg.properties } });
47            break;
48          case MessageType.Event:
49            AppInsights.defaultClient.trackEvent({ name: msg.logSource, properties: msg.properties });
50            break;
51          case MessageType.Exception:
52            AppInsights.defaultClient.trackException({ exception: msg.exception, severity: msg.severity, properties: { source: msg.logSource, ...msg.properties } });
53            break;
54          case MessageType.Metric:
55            AppInsights.defaultClient.trackMetric({ name: msg.logSource, value: +msg.message });
56            break;
57        }
58      }
59    }
```

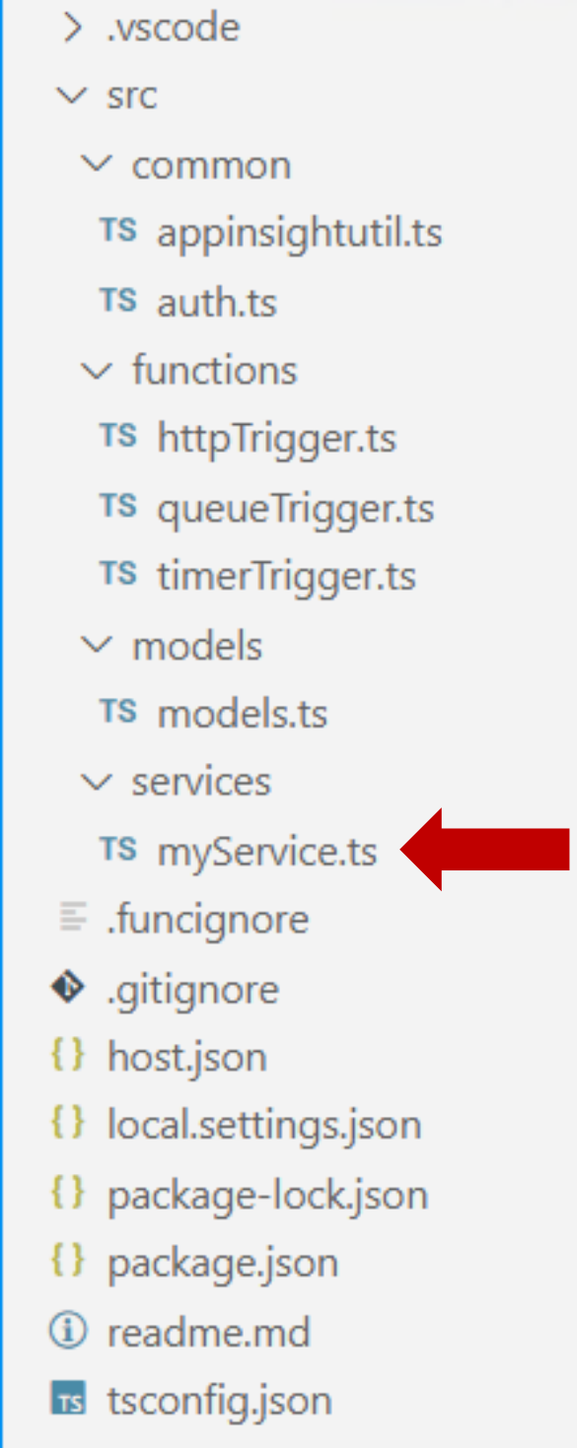
auth.ts

```
1   import { DefaultAzureCredential } from "@azure/identity";
2   import { AzureIdentity } from "@pnp/azidjsclient";
3   import { GraphDefault, SPDefault } from "@pnp/nodejs";
4   import { spfi, SPFI } from "@pnp/sp";
5   import { graphfi, GraphFI } from '@pnp/graph';
6   import { IAppInsightUtil, MessageType, SeverityLevel } from "./appinsightutil.js";
7
8   export interface IAuthService {···
14  }
15
16  export class AuthService implements IAuthService {
17    private LOG_SOURCE = "AuthService";
18    private _ready: boolean = false;
19    private _sp: SPFI = null;
20    private _graph: GraphFI = null;
21    private _spAdmin: SPFI = null;
22    private _apu: IAppInsightUtil = null;
23
24    public constructor(apu: IAppInsightUtil) {
25      this._apu = apu;
26    }
27
28    public async Init(): Promise<boolean> {···
59    }
60
61    public get ready(): boolean {
62      return this._ready;
63    }
64    public get sp(): SPFI {
65      return this._sp;
66    }
67    public get graph(): GraphFI {
68      return this._graph;
69    }
70
71    public get spAdmin(): SPFI {
72      return this._spAdmin
73    }
74  }
```

auth.ts

```
28  public async Init(): Promise<boolean> {
29    let retVal = false;
30    try {
31      const credential = new DefaultAzureCredential();
32      // Create SharePoint credentials
33      this._sp = spfi(process.env.ProjectSite).using(SPDefault(),
34        AzureIdentity(credential, [`https://${process.env.Tenant}.sharepoint.com/.default`], null));
35      // Create Graph credentials
36      this._graph = graphfi().using(GraphDefault(), AzureIdentity(credential, [`https://graph.microsoft.com/.default`], null));
37      // Create SharePoint Admin credentials
38      // const tenantUrl = `https://${process.env.Tenant}-admin.sharepoint.com`;
39      // this._spAdmin = spfi(tenantUrl).using(SPDefault(),
40      // AzureIdentity(credential, [`https://${process.env.Tenant}-admin.sharepoint.com/.default`], null));
41      this._ready = true;
42      this._apu.Log(MessageType.Trace, {
43        message: "Init success",
44        logSource: this.LOG_SOURCE,
45        properties: {
46          method: "Init"
47        },
48        severity: SeverityLevel.Verbose
49      });
50      retVal = true;
51    } catch (err) {
52      this._apu.Log(MessageType.Exception, {
53        logSource: this.LOG_SOURCE,
54        exception: err,
55        severity: SeverityLevel.Critical,
56        properties: {
57          method: "Init"
58        }
59      });
60    }
61    return retVal;
62  }
```

Project Folder



services	Folder that holds any services created for the project
myService.ts	Service that provides the business logic for how to handle changed items.

myService.ts

```ts
import { IAppInsightUtil, MessageType, SeverityLevel } from "../common/appinsightutil";
import { IAuthService } from "../common/auth";
import { IMyQueueItem } from "../models/models";

import "@pnp/sp/webs";
import "@pnp/sp/lists";
import "@pnp/sp/items";

export interface IMyService {
  readonly ready: boolean;
  Init: () => Promise<void>;
  Process: (item: IMyQueueItem) => Promise<boolean>;
  DoJob: () => Promise<boolean>;
}

export class MyService implements IMyService {
  private LOG_SOURCE = "MyService";
  private _ready: boolean = false;

  private _apu: IAppInsightUtil;
  private _auth: IAuthService;

  constructor(apu: IAppInsightUtil, auth: IAuthService) {
    this._apu = apu;
    this._auth = auth;
  }

  public async Init(): Promise<void> {
    this._ready = true;
  }

  public get ready(): boolean {
    return this._ready;
  }

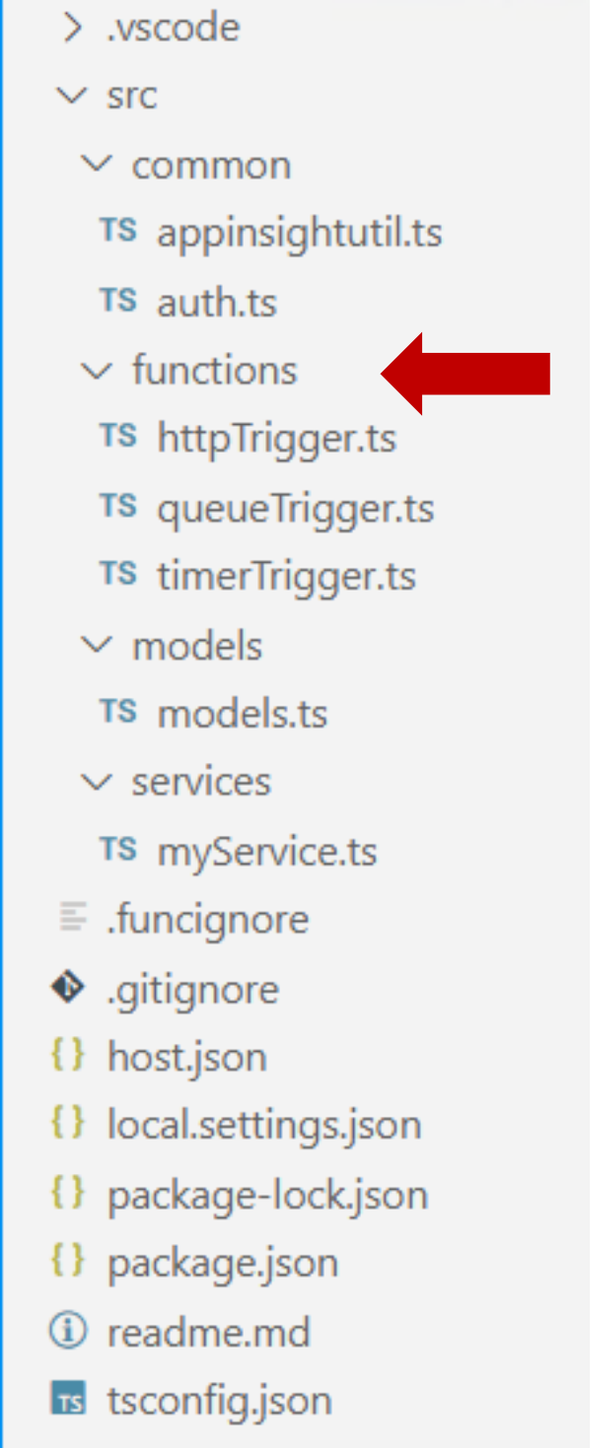
  public async Process(item: IMyQueueItem): Promise<boolean> {···
  }

  public async DoJob(): Promise<boolean> {···
  }
}
```

myService.ts

```
16  export class MyService implements IMyService {
17    private LOG_SOURCE = "MyService";
18    private _ready: boolean = false;
19
20    private _apu: IAppInsightUtil;
21    private _auth: IAuthService;
22
23    constructor(apu: IAppInsightUtil, auth: IAuthService) {···
26    }
27
28    public async Init(): Promise<void> {···
30    }
31
32    public get ready(): boolean {···
34    }
35
36    public async Process(item: IMyQueueItem): Promise<boolean> {
37      let retVal = false;
38      try{
39        // Do something with the queue item.
40        this._auth.sp.web.lists.getById(process.env.ProjectListId).items.getById(item.id).update({Title: 'New Title'});
41        retVal = true;
42      }catch(err){
43        this._apu.Log(MessageType.Exception, {
44          logSource: this.LOG_SOURCE,
45          exception: err,
46          severity: SeverityLevel.Critical,
47          properties: {
48            method: "Process"
49          }
50        });
51      }
52      return retVal;
53    }
54
55    public async DoJob(): Promise<boolean> {···
71    }
72  }
73
```

Project Folder



functions	Folder that holds all of the trigger definitions for the Azure Function
httpTrigger.ts	Can be configured to respond to various fetch commands
queueTrigger.ts	Triggers when item is added to the Azure Storage queue it's been configured to watch.
timerTrigger.ts	Triggers based on CRON settings.

timerTrigger.ts

```
1   import { app, InvocationContext, Timer } from "@azure/functions";
2   import { AppInsightUtil, MessageType, SeverityLevel } from "../common/appinsightutil.js";
3   import { AuthService } from "../common/auth.js";
4   import { MyService } from "../services/myService.js";
5
6   export async function timerTrigger(myTimer: Timer, context: InvocationContext): Promise<void> {
7     const LOG_SOURCE = "TimerTrigger";
8     const _apu = new AppInsightUtil();
      _apu.Init(context.invocationId);
10
11    _apu.Log(MessageType.Event, {
12      logSource: `${LOG_SOURCE}`,
13      properties: {
14        source: LOG_SOURCE,
15        message: "Starting notification queue trigger"
16      }
17    });
18
      try {···
50  >   } catch (err) {···
56    }
57  }
58
    app.timer('timerTrigger', {
60      schedule: process.env.SCHEDULE,
61      runOnStartup: process.env.DEBUG === 'true' ? true : false,
62      handler: timerTrigger
63  });
```

timerTrigger.ts

```
6   export async function timerTrigger(myTimer: Timer, context: InvocationContext): Promise<void> {
7     const LOG_SOURCE = "TimerTrigger";
8     const _apu = new AppInsightUtil();
9     _apu.Init(context.invocationId);
10
11  >  _apu.Log(MessageType.Event, {···
17    });
18
19    try {
20      const timeStamp = new Date().toISOString();
21      if (myTimer.isPastDue) {
22  >     _apu.Log(MessageType.Trace, {···
29        });
30      }
31
32      const auth = new AuthService(_apu);
33      const initialized = await auth.Init();
34      let result = false;
35      if (initialized) {
36        const myService = new MyService(_apu, auth);
37        result = await myService.DoJob();
38      }
39
40  >   _apu.Log(MessageType.Trace, {···
49      });
50    } catch (err) {
51  >   _apu.Log(MessageType.Exception, {···
55      });
56    }
57  }
```

httpTrigger.ts

```
1   import { app, HttpRequest, HttpResponseInit, InvocationContext, output } from "@azure/functions";
2   import { AppInsightUtil, MessageType, SeverityLevel } from "../common/appinsightutil.js";
3   import { IMyQueueItem } from "../models/models.js";
4
5   const queueOutput = output.storageQueue({
6     queueName: process.env.MyQueuePath,
7     connection: 'AzureWebJobsStorage',
8   });
9
10  export async function httpRequest(request: HttpRequest, context: InvocationContext): Promise<HttpResponseInit> {···
38  };
39
40  app.http('httpRequest', {
41    methods: ['POST'],
42    authLevel: 'anonymous',
43    extraOutputs: [queueOutput],
44    handler: httpRequest,
45  });
```

httpTrigger.ts

```
10  export async function httpRequest(request: HttpRequest, context: InvocationContext): Promise<HttpResponseInit> {
11    const LOG_SOURCE = "provisionCompleteHttp";
12    const _apu = new AppInsightUtil();
13    _apu.Init(context.invocationId);
14
15  >   _apu.Log(MessageType.Event, {···
21    });
22
23    try {
24      var requestBody = await request.json();
25      var data: IMyQueueItem = requestBody as IMyQueueItem;
26      context.extraOutputs.set(queueOutput, JSON.stringify(data));
27      return { status: 200 };
28    } catch (err) {
29  >     _apu.Log(MessageType.Exception, {···
36      });
37      return { status: 500, body: JSON.stringify({ message: `Critical error - ${err}` }) };
38    }
39  };
```

queueTrigger.ts

```
1   import { app, InvocationContext } from "@azure/functions";
2   import { AppInsightUtil, MessageType, SeverityLevel } from "../common/appinsightutil.js";
3   import { AuthService } from "../common/auth.js";
4   import { IMyQueueItem } from "../models/models.js";
5   import { MyService } from "../services/myService.js";
6
    export async function queueTrigger(queueItem: IMyQueueItem, context: InvocationContext): Promise<void> {···
51  }
52
    app.storageQueue('queueTrigger', {
54    queueName: process.env.MyQueuePath,
55    connection: 'AzureWebJobsStorage',
56    handler: queueTrigger
57  });
```

queueTrigger.ts

```
8   export async function queueTrigger(queueItem: IMyQueueItem, context: InvocationContext): Promise<void> {
9     const LOG_SOURCE = "provisionQueue";
10    const _apu = new AppInsightUtil();
11    _apu.Init(context.invocationId);
12
13  >   _apu.Log(MessageType.Event, {···
19    });
20
21    try {
22      const auth = new AuthService(_apu);
23      const initialized = await auth.Init();
24      let result = false;
25      if (initialized) {
26        const myService = new MyService(_apu, auth);
27        result = await myService.Process(queueItem);
28      }
29  >   _apu.Log(MessageType.Trace, {···
37      });
38      // Used to throw exception in AppInsights and will be requeued based on settings in host.json
39  >   if (!result) {···
41      }
42      // Alternate: Requeue logic
43  >   if (!result) {···
72      }
73    } catch (err) {
74  >   _apu.Log(MessageType.Exception, {···
81      });
82      throw Error(`Queue item was not processed. ${err}`);
83    }
84  }
```

Using Queues – Requeuing Option 1

```
38      // Used to throw exception in AppInsights and will be requeued based on settings in host.json
39      if (!result) {
40        throw Error(`Error processing queue item ${JSON.stringify(queueItem)}`);
41      }
42      // Alternate: Requeue logic
43    > if (!result) {···
72      }
73    } catch (err) {
74      _apu.Log(MessageType.Exception, {
75        logSource: LOG_SOURCE,
76        exception: err,
77        severity: SeverityLevel.Critical,
78        properties: {
79          method: "notificationQueue"
80        }
81      });
82      throw Error(`Queue item was not processed. ${err}`);
83    }
```

Using Queues – Requeuing Option 2





```
// Alternate: Requeue logic
if (!result) {
  const queueClient = new QueueClient(
    process.env.AzureWebJobsStorage,
    process.env.MyQueuePath
  );

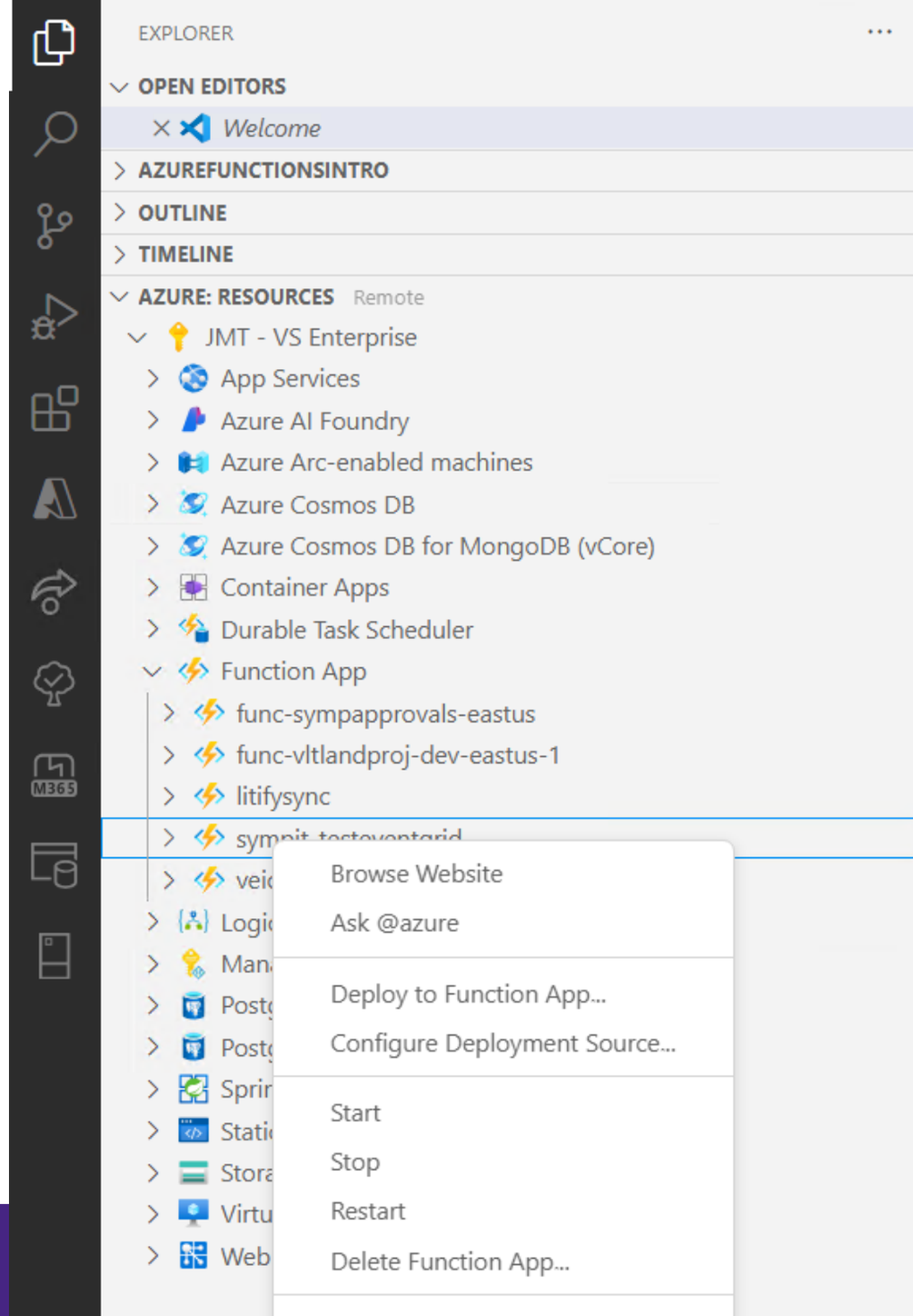
  const response = await queueClient.updateMessage(
    context.triggerMetadata.id as string,
    context.triggerMetadata.popReceipt as string,
    Buffer.from(JSON.stringify(queueItem)).toString("base64"),
    300
  );

  _apu.Log(MessageType.Trace, {
    message: `Requeued Item - Incomplete`,
    logSource: LOG_SOURCE,
    properties: {
      method: "{function}",
      result: response.errorCode || "Success",
      messageId: context.triggerMetadata.id,
      popReceipt: context.triggerMetadata.popReceipt,
      queueItem: JSON.stringify(queueItem)
    },
    severity: (response.errorCode) ? SeverityLevel.Error : SeverityLevel.Information
  });

  if (response.errorCode) {
    throw Error(`Could not requeue item.`);
  }
```

Deployment

- Right-Click deploy with Azure Tools extension.
- Create CI/CD pipeline with Azure DevOps or GitHub Actions
- Add IaC (Infrastructure as Code) with BICEP files defining your Azure resources.



Summary

- Discussed common use cases and architectural patterns
- Discussed security, including MSAL, Azure Identity, and OBO Flows
- Created a NodeJS project and configured it
- Added helper functions for logging and authentication
- Reviewed some of the most common trigger types
- Discussed separation of business logic
- Quick review of deployment and automation

Resources

- VSC Extension: Azure Tools
- VSC Extension: Azure Functions
- Partner Events overview for customers - Azure Event Grid
- @azure/identity NPM Package
- What are managed identities for Azure resources?
- How to: CI/CD/IaC for Azure Function Apps and GitHub Actions